

K8s Guardian Project Blueprint

K8s Guardian — Policy-Enforced, Secure GitOps Deployment Engine

Project positioning (for your resume/README)

Designed and built a policy-enforced Kubernetes deployment platform that blocks unsafe workloads at admission, manages secrets securely, continuously reconciles infrastructure through GitOps, enforces zero-downtime rollouts, and automatically recovers from and alerts on runtime failures.

This sits alongside your existing **Poka-Yoke DevSecOps Gatekeeper** project as a natural second half:

- **Poka-Yoke Gatekeeper** → “I prevent bad code from passing CI.”
- **K8s Guardian** → “I prevent bad workloads from running, keep secrets safe, keep the cluster in sync with Git, and alert when something breaks.”

Together they tell one coherent DevSecOps story instead of two disconnected projects.

What changed from the original blueprint

Added (closes the “not quite production-grade” gaps): 1. **Secrets management** — Sealed Secrets instead of plain K8s Secrets 2. **PodDisruptionBudget** — protects availability during node drains/upgrades 3. **Alertmanager** — turns dashboards into actual alerts 4. **Kustomize overlays (dev/prod)** — multi-environment story, not just one cluster 5. **Image signing + SBOM (cosign + Syft)** — supply-chain security, a step beyond Trivy scanning alone 6. **Centralized logging (Loki + Promtail)** — logs alongside metrics, not metrics-only

Removed/deferred: - EKS/Terraform extension (you already have AWS depth on your resume — this project’s job is to prove Kubernetes depth, not repeat AWS) - 28-phase pacing collapsed into **4 build tiers**, so you have a resume-worthy, demoable project much sooner, with clear stopping points if time runs out

Build tiers (stop at any tier and still have something real to show)

Tier 1 — Core Kubernetes (resume-ready baseline, ~1 week)

Working containerized app running reliably on Kubernetes.

- ☐ Flask/FastAPI app with `/`, `/healthz`, `/ready`, `/kill`, `/version` endpoints
- ☐ Dockerfile: non-root user, pinned base image, minimal layers
- ☐ `kind` cluster (control-plane + 2 workers)
- ☐ Deployment (3 replicas), Service, ConfigMap
- ☐ `livenessProbe`, `readinessProbe`, `startupProbe`
- ☐ `resources.requests` / `resources.limits`
- ☐ RollingUpdate strategy: `maxUnavailable: 0`, `maxSurge: 25%`
- ☐ **PodDisruptionBudget** (`minAvailable: 2`)
- ☐ Demonstrate: pod crash → restart; pod delete → replacement; rolling update → zero dropped requests

Stopping here already gives you: “Deployed and hardened a self-healing Kubernetes workload with zero-downtime rollouts and availability guarantees during disruptions.”

Tier 2 — Policy + Secrets + GitOps (~1 more week)

This is where it becomes a *Guardian*, not just a deployment.

- ☐ Install Kyverno; policies: no privileged, no `:latest`, mandatory resources, non-root, mandatory probes, read-only root filesystem
- ☐ **Sealed Secrets** — encrypt a DB credential, commit the sealed version to Git safely, controller decrypts in-cluster
- ☐ Argo CD Application pointing at your Git repo, `automated: { prune: true, selfHeal: true }`
- ☐ Demonstrate **drift reconciliation**: manually `kubectl scale --replicas=1` → Argo CD reverts to Git's `replicas: 3`
- ☐ Demonstrate **admission rejection**: apply a privileged/ `:latest` manifest → blocked before it ever runs
- ☐ `tests/` folder with `bad-privileged.yaml`, `bad-latest.yaml`, `bad-root.yaml`, `compliant.yaml` + a results table in the README

Stopping here already gives you: “Implemented policy-as-code admission control and GitOps-based continuous reconciliation, with encrypted secrets management.”

Tier 3 — CI/CD + Supply Chain + Scaling (~1 more week)

Full pipeline, image trust, automatic scaling.

- ☐ GitHub Actions: lint → unit tests → Trivy scan → Docker build → **Syft SBOM** → **cosign image signing** → push to GHCR → bump image tag in the GitOps repo (commit, not `kubectl apply`)
- ☐ Immutable, semantic image tags (`v1.0.0`, never `latest`) — enforced twice: by convention in CI and by Kyverno at admission
- ☐ Metrics Server + HPA (min 2 / max 10, CPU target 60%)
- ☐ Load test → watch scale-out → stop load → watch scale-in
- ☐ **Kustomize overlays**: `base/` + `overlays/dev/` + `overlays/prod/` (different replica counts, resource limits, HPA thresholds per environment)

Stopping here already gives you: “Built a full CI/CD pipeline with image signing and SBOM generation, and Kubernetes-native autoscaling across dev/prod environments.”

Tier 4 — Observability + Network Security (stretch, if time allows)

Makes the project demo-video-ready and interview-proof.

- ☐ Prometheus + Grafana: pod count, CPU/memory, restarts, request rate, HPA activity
- ☐ **Loki + Promtail** for centralized log aggregation (correlate a pod restart with its logs in one place)
- ☐ **Alertmanager** rule: fire an alert when restart count > N in 5 minutes (route to a webhook/Slack — even a dummy one is fine for a demo)
- ☐ Ingress with a local hostname
- ☐ NetworkPolicy: frontend → backend → database only, deny-by-default elsewhere
- ☐ Dedicated ServiceAccount + RBAC Role/RoleBinding scoped to only what the app needs
- ☐ Record a 3–5 min demo video walking through: admission rejection → GitOps drift fix → rolling update → HPA scale → alert firing

Final failure/behavior matrix (your README's acceptance criteria)

Scenario	Mechanism	Expected behavior
Privileged container	Kyverno	✗ Rejected at admission
<code>:latest</code> tag	Kyverno	✗ Rejected at admission
Missing resource limits	Kyverno	✗ Rejected at admission
Plaintext secret in Git	Sealed Secrets	✗ Not possible — only sealed form is committed

Pod crashes	Liveness probe	🔄 Restarted
Pod deleted	Deployment controller	🔄 Replaced
Pod not ready	Readiness probe	🚫 Removed from Service endpoints
Node drain / disruption	PodDisruptionBudget	✔️ minAvailable maintained
High CPU	HPA	📈 Scale out
Low CPU	HPA	📉 Scale in
Manual cluster drift	Argo CD	🔄 Reconciled back to Git state
New version release	RollingUpdate (maxUnavailable: 0)	✔️ Zero-downtime rollout
Unsigned/unscanned image	CI pipeline	❌ Pipeline fails before push
Unauthorized pod-to-pod traffic	NetworkPolicy	🚫 Blocked
Repeated crash loop	Alertmanager	🔔 Alert fires

This table doubles as your project’s test plan and your resume bullet source material.

Suggested resume bullets once built

- “Architected a policy-enforced Kubernetes platform using Kyverno admission control to block privileged, unpatched, or non-compliant workloads before deployment.”
- “Implemented GitOps continuous delivery with Argo CD, including automated drift detection and self-healing reconciliation between Git and cluster state.”
- “Secured application secrets using Sealed Secrets, enabling safe secret storage in version control without exposing plaintext credentials.”
- “Built a CI/CD pipeline with GitHub Actions incorporating Trivy vulnerability scanning, SBOM generation, and cosign image signing for supply-chain security.”
- “Configured Horizontal Pod Autoscaling, PodDisruptionBudgets, and zero-downtime rolling updates to maintain availability under load and during disruptions.”
- “Deployed centralized observability using Prometheus, Grafana, and Loki, with Alertmanager rules for automated incident detection.”

Suggested repo structure

```

k8s-guardian/
├── app/
│   ├── src/main.py
│   ├── requirements.txt
│   └── Dockerfile
├── k8s/
│   ├── base/
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   ├── configmap.yaml
│   │   ├── ingress.yaml
│   │   ├── hpa.yaml
│   │   ├── pdb.yaml
│   │   ├── serviceaccount.yaml
│   │   ├── role.yaml
│   │   ├── rolebinding.yaml
│   │   └── networkpolicy.yaml
│   ├── overlays/
│   │   ├── dev/kustomization.yaml
│   │   └── prod/kustomization.yaml
│   ├── policies/           # Kyverno ClusterPolicies
│   └── secrets/             # SealedSecret manifests (safe to commit)

```

```
├── argocd/
│   ├── application.yaml
│   └── project.yaml
├── monitoring/
│   ├── prometheus/
│   ├── grafana/
│   ├── loki/
│   └── alertmanager/
├── tests/
│   ├── admission/           # bad-privileged.yaml, bad-latest.yaml, etc.
│   ├── reliability/        # pod recovery, readiness, rolling update scripts
│   └── gitops/             # drift reconciliation script
├── .github/workflows/
│   ├── ci.yaml
│   └── security.yaml
├── kind-cluster.yaml
├── Makefile
└── README.md
```

Realistic pacing

- **Tier 1:** ~5–7 evenings — do this first, no shortcuts, this is your foundation
- **Tier 2:** ~5–7 evenings — this is the “wow” tier, prioritize this over Tier 4
- **Tier 3:** ~5–7 evenings
- **Tier 4:** whatever time is left before you need it on your resume

If you’re applying soon, **Tier 1 + Tier 2 alone is already a strong, defensible resume project** — don’t feel you need all four tiers before it’s “good enough” to list.